

API Authentication Guide

wiki/engineering/api-auth-guide · Updated 2026-06-01 · Owner: Platform

Overview

This guide explains how external clients authenticate with the Northwind API. It is maintained by the Platform team and is the reference for partners and internal teams building integrations.

Authentication is the front door to customer data, so the API uses a standard, well understood scheme and enforces transport security everywhere.

Concepts

A few terms are used throughout this guide. A client is an application that calls the API. A client identity is the credential that identifies that application. An access token is a short-lived credential the client presents on each request. A scope is a permission that limits what a token can do.

Authentication model

External clients authenticate using OAuth 2.0 client credentials. Each integration is issued its own client identity so access can be granted and revoked independently.

- Request a client ID and secret from the developer portal.

- Exchange the client credentials for an access token at the token endpoint.
- Send the access token as a bearer token on every API request.

Getting credentials

Register your integration in the developer portal to receive a client ID and secret. Use a separate client identity for each environment and each distinct integration so you can revoke one without affecting the others.

Token lifetime

Access tokens expire after one hour; refresh using the token endpoint. Build your client to refresh tokens automatically shortly before they expire rather than waiting for a request to fail. Cache the token and reuse it until it is close to expiry; do not request a new token on every call.

Handling secrets

Client secrets are sensitive and must be protected.

- Never embed secrets in frontend code or mobile apps.
- Store secrets in a secret manager or secure server-side configuration.
- Rotate secrets periodically and immediately if you suspect exposure.
- Use a separate client identity per environment.

Rotating a secret

To rotate without downtime, create a second secret for the same client, deploy it, confirm the new secret works, then retire the old one. Never rotate

by deleting the only working secret first.

Transport security

All API traffic must use TLS 1.2 or higher. Requests over plain HTTP are rejected. Verify server certificates and do not disable certificate validation, even in test environments.

Scopes and least privilege

Each client is granted only the scopes it needs. Request the minimum set of scopes for your integration, and request additional scopes only when a new feature requires them. Over-broad access is a common source of security findings.

Reviewing scopes

Review the scopes your integration holds periodically and give up any you no longer use. The fewer permissions a leaked token carries, the less damage it can do.

Error handling and rate limits

- A 401 response means the token is missing, expired, or invalid; refresh and retry once.
- A 403 response means the token is valid but lacks the required scope.
- A 429 response means you are being rate limited; back off and retry with jitter.

Retries

Use exponential backoff with jitter for transient errors. Do not retry non-idempotent requests blindly, and never retry a 4xx that indicates a client mistake without fixing the request first.

Worked example

A partner builds a server-side integration. They register a client in the developer portal for the staging environment, store the secret in their secret manager, and request only the read scopes they need. Their client fetches an access token, caches it, and refreshes it a few minutes before the one-hour expiry. On a 429 they back off with jitter and retry. When they go live they register a separate production client.

Frequently asked questions

How long do tokens last? One hour. Refresh at the token endpoint before expiry.

Where do I put my secret? In a server-side secret manager, never in frontend or mobile code.

What does a 403 mean? Your token is valid but lacks the required scope.

Common mistakes

- Embedding a client secret in a mobile app or single-page frontend.
- Requesting a new token on every API call instead of caching it.

- Requesting far more scopes than the integration needs.
- Disabling certificate validation to "make it work" in testing.

Support

For client credentials, scope changes, or integration help, contact the Platform team through the developer portal. Include your client ID but never your secret.

Related documents

See the Updated Security Standards for secret handling, and the CI/CD Pipeline Standards for how services manage configuration and secrets at deploy time.

Testing your integration

Test against the staging environment with a dedicated staging client before going live. This lets you exercise the full flow without touching production data or limits.

- Register a separate client identity for staging and never reuse production secrets there.
- Verify that your client fetches a token, caches it, and refreshes it before the one-hour expiry.
- Confirm that your error handling behaves correctly for 401, 403, and 429 responses.

- Check that your client only requests the scopes it actually needs.

Once staging looks correct, register a production client, move the secret into your production secret store, and repeat a brief smoke test before cutting over real traffic.